

Árboles de decisión

Guía introductoria reproducible

Conceptos, código, entradas, salidas e interpretación

Caso conductor: clasificación de riesgo con 480 casos completamente simulados. Incluye un notebook ejecutable y un flujo que regenera todas las figuras y métricas.

Cómo usar esta guía

Este texto está diseñado para una exposición introductoria de 12 a 15 minutos y para una práctica posterior. La lectura principal explica el razonamiento; el notebook `notebooks/01_arboles_decision.ipynb` permite ejecutar cada paso y observar sus salidas.

Ruta de aprendizaje

Pregunta → división → árbol → predicción → evaluación → uso responsable.

Alcance

Los datos son sintéticos y no representan pacientes reales. El ejemplo enseña un flujo de ciencia de datos; no valida una regla clínica ni autoriza decisiones sobre personas.

Índice general

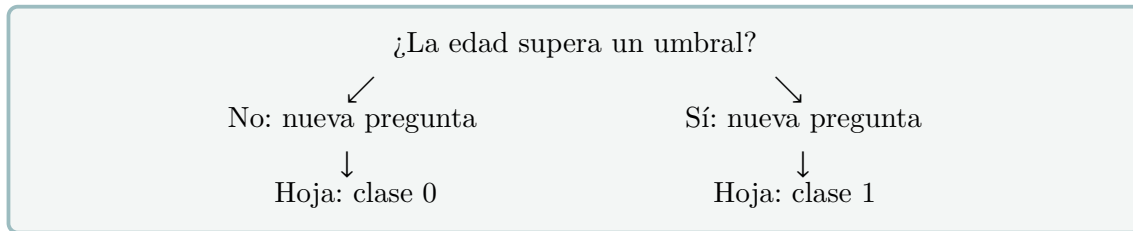
Cómo usar esta guía	I
1. Una idea sencilla: aprender preguntas	1
1.1. Clasificación y regresión	1
1.2. Anatomía mínima	1
2. Cómo elige el árbol una división	3
2.1. Entropía	3
3. Diseño reproducible del ejemplo	5
3.1. Datos de entrada	5
3.2. Separación entrenamiento/prueba	6
3.3. Modelo deliberadamente pequeño	6
4. El árbol y sus reglas	7
5. Entradas, salidas y evaluación	9
5.1. Resultados reproducidos	9
5.2. Matriz de confusión	9
5.3. Ejemplos de entrada y salida	10
6. Complejidad, poda y sobreajuste	12
7. Interpretación responsable	13
7.1. Ventajas	13
7.2. Limitaciones	14
8. Guion breve para exponer	15
8.1. Preguntas frecuentes	15

9. Reproducir el proyecto	16
Referencias y lecturas	17
Lista de comprobación final	18

Capítulo 1

Una idea sencilla: aprender preguntas

Un árbol de decisión representa una predicción como una secuencia de preguntas. Cada pregunta divide los datos en grupos cada vez más homogéneos. Al final de una ruta aparece una **hoja**, que contiene la predicción.



1.1. Clasificación y regresión

La estructura es la misma, pero cambia el tipo de respuesta:

	Clasificación	Regresión
Respuesta	Categoría, por ejemplo riesgo alto/bajo	Cantidad continua, por ejemplo costo o concentración
Criterio típico	Gini o entropía	Error cuadrático
Salida de una hoja	Clase y proporción estimada	Promedio de los casos de la hoja

Nuestro caso es de clasificación binaria: riesgo_alto vale 0 o 1.

1.2. Anatomía mínima

Nodo raíz

contiene todos los casos de entrenamiento y realiza la primera pregunta.

Nodo interno

realiza otra pregunta después de una división previa.

Rama

representa la respuesta verdadera o falsa a una pregunta.

Hoja

termina la ruta y entrega una predicción.

Profundidad

cuenta cuántas preguntas separan la raíz de una hoja.

Capítulo 2

Cómo elige el árbol una división

Para cada variable y varios umbrales candidatos, el algoritmo mide qué tan bien separaría las clases. En este proyecto usamos la impureza de Gini:

$$G(t) = 1 - \sum_{k=1}^K p_{k,t}^2, \quad (2.1)$$

donde $p_{k,t}$ es la proporción de la clase k dentro del nodo t . Un nodo puro tiene $G = 0$. En un problema binario, una mezcla 50/50 tiene $G = 0.5$.

Para comparar una división se ponderan sus dos hijos:

$$G_{\text{div}} = \frac{n_L}{n} G(L) + \frac{n_R}{n} G(R). \quad (2.2)$$

El árbol selecciona la pregunta que más reduce la impureza respecto al nodo original. Después repite el proceso dentro de cada hijo.

```
def gini_impurity(labels):
    values, counts = np.unique(labels, return_counts=True)
    probabilities = counts / counts.sum()
    return 1.0 - np.square(probabilities).sum()
```

Ejemplo mental

Un nodo con clases $[0, 0, 1, 1]$ tiene Gini 0.5. Si una pregunta crea dos hijos $[0, 0]$ y $[1, 1]$, ambos son puros y el Gini ponderado es 0. Esa pregunta separa perfectamente esos ocho elementos conceptuales.

2.1. Entropía

Otra medida habitual es

$$H(t) = - \sum_{k=1}^K p_{k,t} \log_2 p_{k,t}. \quad (2.3)$$

Gini y entropía suelen producir árboles parecidos. Para una introducción es más importante comprender que ambas premian hijos homogéneos que memorizar cuál usar.

Capítulo 3

Diseño reproducible del ejemplo

3.1. Datos de entrada

El script `scripts/generate_data.py` crea 480 casos con una semilla fija. Contiene seis predictores:

Variable	Interpretación pedagógica
edad	años simulados
presion_sistolica	medida simulada en mmHg
colesterol	medida simulada en mg/dL
fumador	indicador binario simulado
actividad_horas_semana	actividad semanal simulada
antecedente_familiar	indicador binario simulado

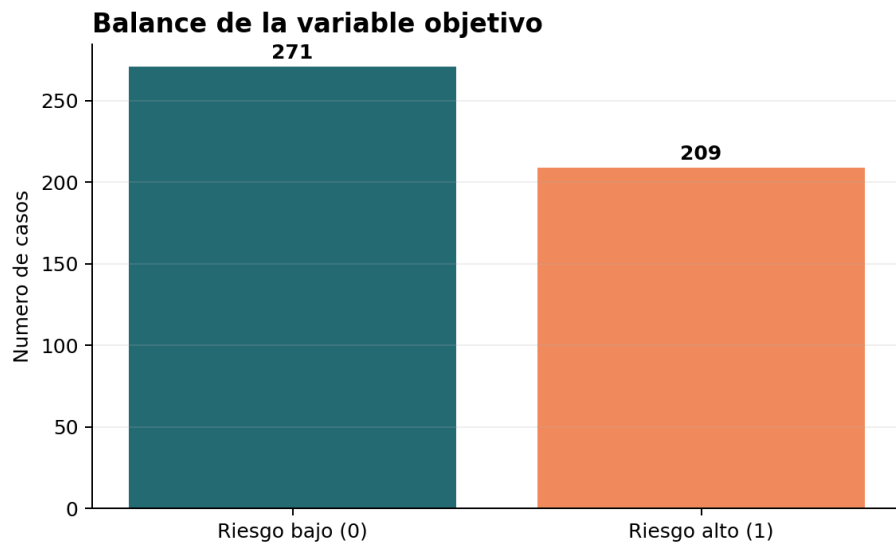


Figura 3.1: Distribución de la respuesta en los datos simulados.

3.2. Separación entrenamiento/prueba

Antes de ajustar el árbol reservamos 120 observaciones (25 %) para prueba y usamos 360 para entrenamiento. La separación es estratificada, por lo que conserva aproximadamente la proporción de clases.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)
```

Fuga de información

El conjunto de prueba simula datos nuevos. No debe utilizarse para escoger variables, crear la respuesta, ajustar parámetros ni decidir cuándo detener el entrenamiento. Si se usa repetidamente para tomar decisiones, deja de ser una evaluación final independiente.

3.3. Modelo deliberadamente pequeño

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    min_samples_leaf=20,
    random_state=42,
)
model.fit(X_train, y_train)
```

El árbol resultante tiene profundidad 3 y 8 hojas. La restricción favorece una explicación clara, pero en una aplicación real estos valores deben justificarse mediante validación y conocimiento del problema.

Capítulo 4

El árbol y sus reglas

Para leer un nodo:

1. comprobar la pregunta de la primera línea;
2. ir a la izquierda si es verdadera y a la derecha si es falsa;
3. observar samples y value para saber cuántos casos llegaron;
4. continuar hasta una hoja y leer la clase mayoritaria.

Regla, no explicación causal

Una ruta describe cómo este modelo asigna una predicción. No prueba que cambiar una variable cause un cambio en el desenlace. Asociación predictiva y causalidad son preguntas diferentes.

Arbol entrenado: profundidad maxima = 3

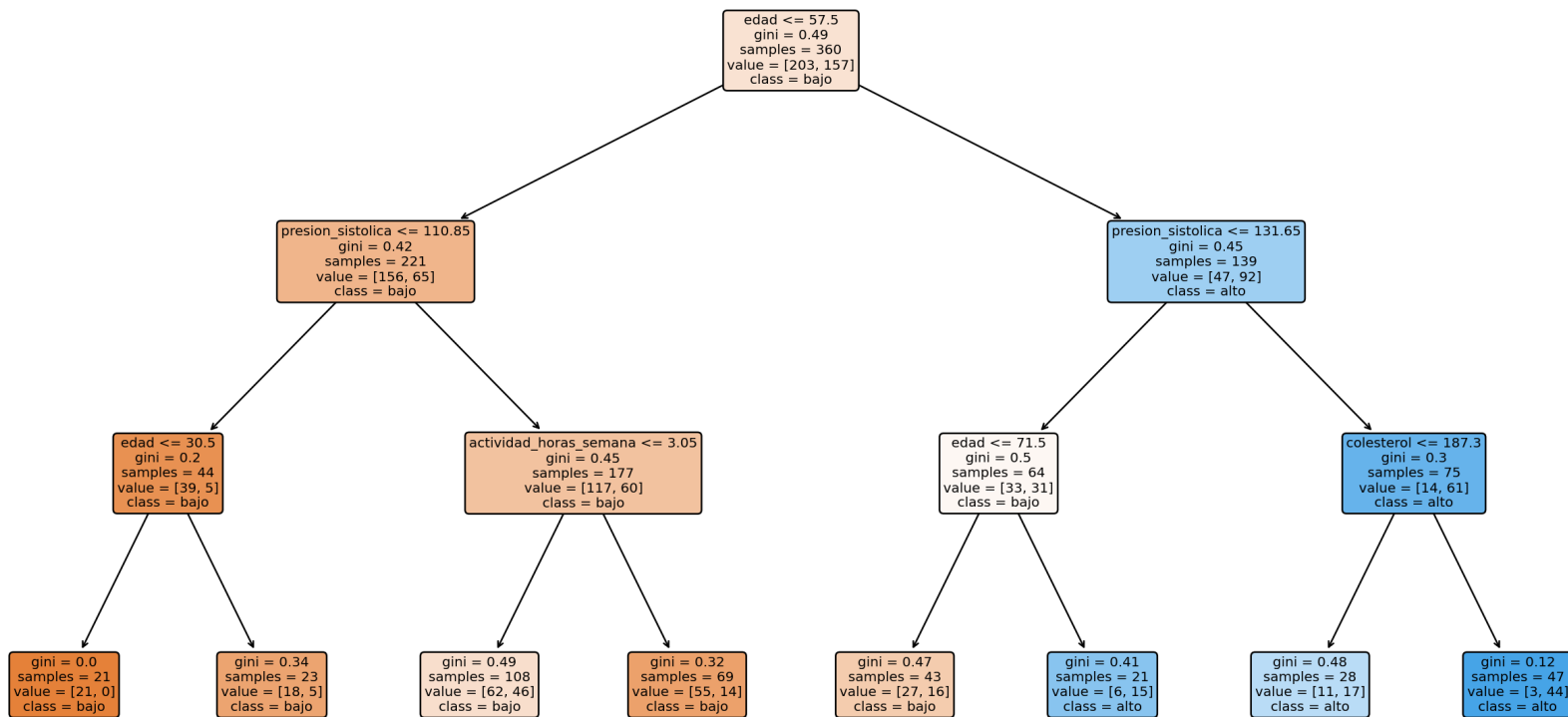


Figura 4.1: Árbol aprendido únicamente con el conjunto de entrenamiento. Los colores indican la clase dominante, no certeza causal.

Capítulo 5

Entradas, salidas y evaluación

El siguiente fragmento produce probabilidades, clases y métricas sobre el conjunto que el árbol no vio durante el ajuste:

```
prediction = model.predict(X_test)
probability = model.predict_proba(X_test)[:, 1]

accuracy = accuracy_score(y_test, prediction)
precision = precision_score(y_test, prediction)
recall = recall_score(y_test, prediction)
roc_auc = roc_auc_score(y_test, probability)
```

5.1. Resultados reproducidos

Métrica	Valor en prueba
Exactitud	0.733
Precisión	0.794
Sensibilidad (recall)	0.519
F1	0.628
ROC AUC	0.724
Referencia: clase mayoritaria	0.567

La exactitud supera la referencia ingenua que siempre predeciría la clase más frecuente. Aun así, la sensibilidad muestra que algunos casos positivos no son detectados. El modelo es útil para enseñar la evaluación precisamente porque no es perfecto.

5.2. Matriz de confusión

	Predicho 0	Predicho 1
Observado 0	61	7
Observado 1	25	27

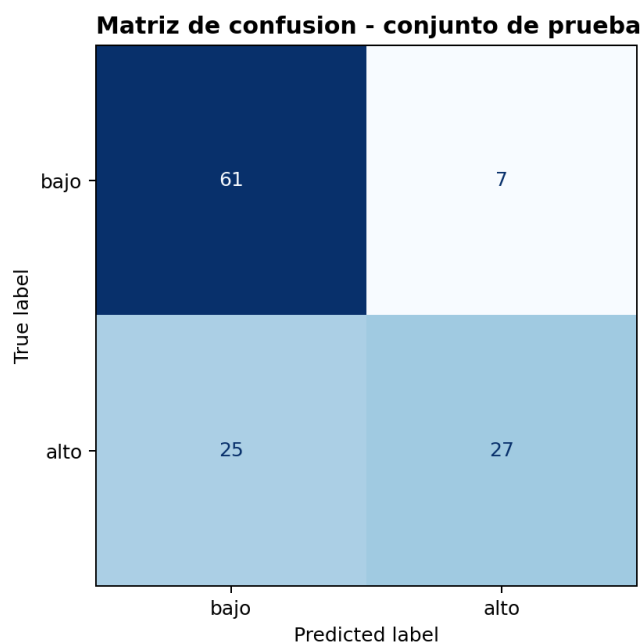


Figura 5.1: Cada celda corresponde a un tipo de acierto o error.

5.3. Ejemplos de entrada y salida

Estas filas proceden del CSV de predicciones generado por el pipeline:

ID	Edad	PAS	Fuma	Obs.	Pred.	$P(y = 1)$
10	69	129.9	0	0	0	0.372
11	50	134.9	1	0	0	0.426
19	41	125.6	0	0	0	0.426
21	51	152.2	0	0	0	0.426
24	32	148.5	0	0	0	0.426
28	23	113.7	0	0	0	0.426

La probabilidad es la proporción de clase 1 observada en la hoja correspondiente. En árboles pequeños puede repetirse para muchos casos y no debe interpretarse automáticamente como una probabilidad clínicamente calibrada.

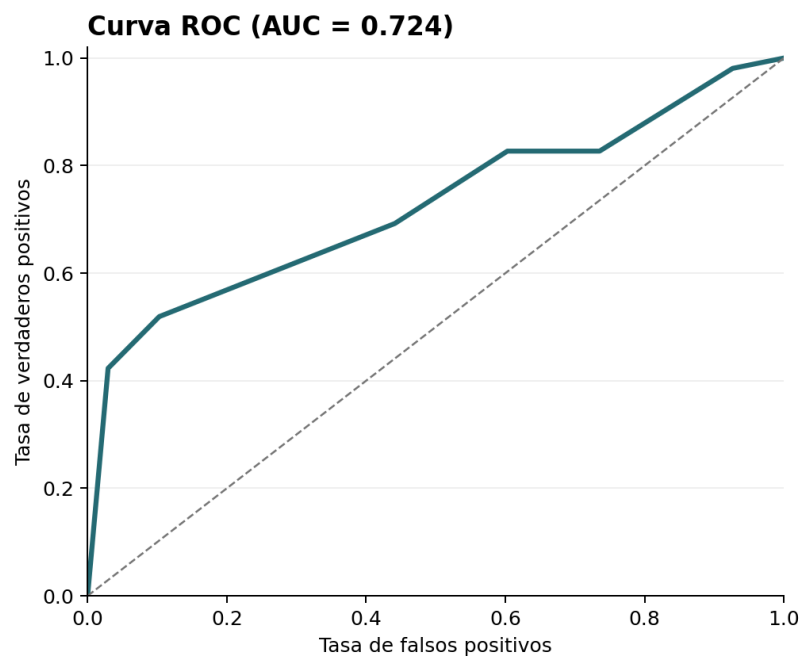


Figura 5.2: La curva ROC resume el intercambio entre sensibilidad y falsos positivos a través de varios umbrales.

Capítulo 6

Complejidad, poda y sobreajuste

Un árbol sin restricciones puede continuar dividiendo hasta memorizar detalles del entrenamiento. Esto reduce el error aparente, pero las reglas pueden ser inestables y generalizar peor.

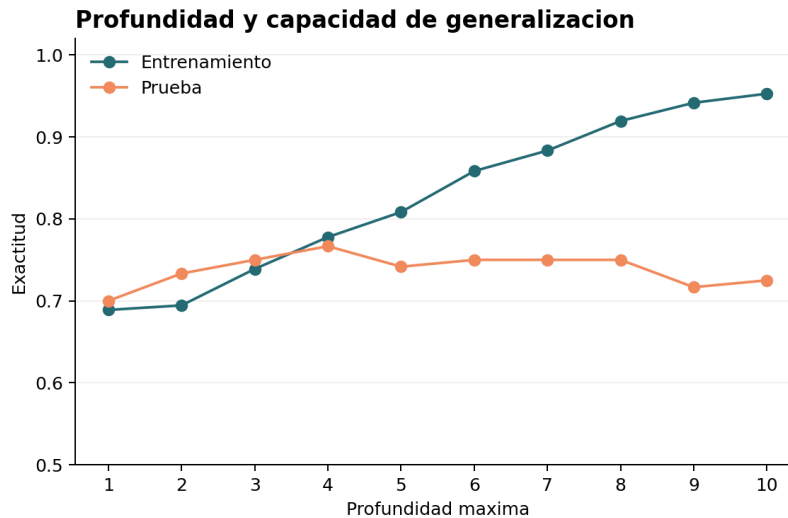


Figura 6.1: La exactitud de entrenamiento crece con la profundidad; la de prueba no tiene por qué hacerlo. Esta comparación es ilustrativa, no sustituye la validación cruzada.

Dos familias de control son:

Pre-poda

limitar profundidad, número de hojas o tamaño mínimo de hoja antes y durante el crecimiento.

Post-poda

crecer un árbol y retirar ramas cuyo beneficio no compensa su complejidad. CART formaliza esta idea mediante poda por costo-complejidad.

Una forma conceptual del criterio es

$$R_{\alpha}(T) = R(T) + \alpha|T|, \quad (6.1)$$

donde $R(T)$ mide el error, $|T|$ la cantidad de hojas y α penaliza la complejidad. Un α mayor favorece árboles más pequeños.

Capítulo 7

Interpretación responsable

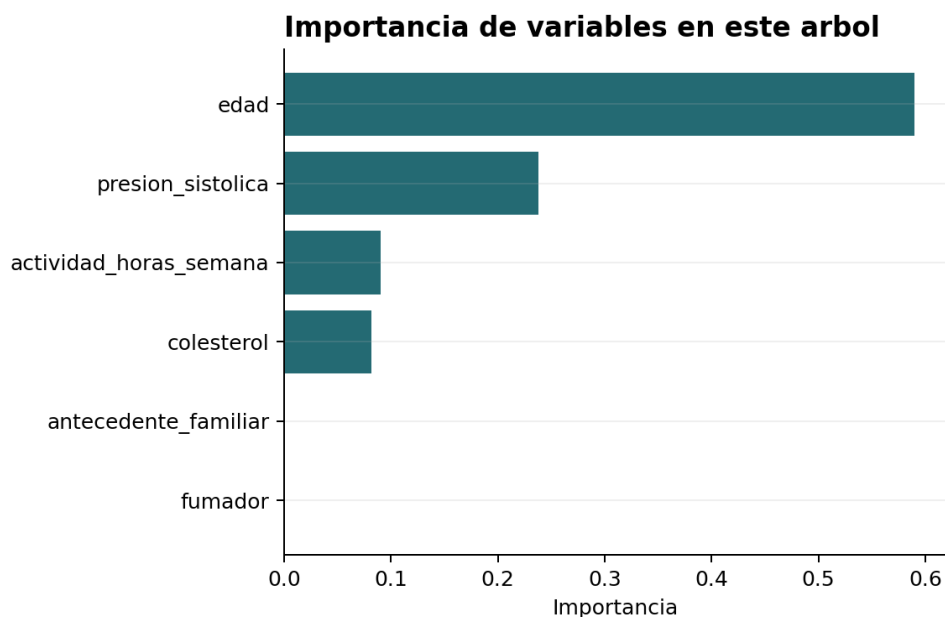


Figura 7.1: Importancia basada en reducción de impureza dentro de este modelo.

La importancia indica cuánto contribuyó una variable a reducir la impureza en este árbol. No representa un efecto causal y puede favorecer variables con más oportunidades de división. Las variables correlacionadas también pueden repartirse o intercambiar importancia.

7.1. Ventajas

- reglas visuales y fáciles de comunicar;
- relaciones no lineales e interacciones sin especificarlas a mano;
- poca preparación numérica para un ejemplo tabular limpio;
- aplicable a clasificación y regresión.

7.2. Limitaciones

- alta variabilidad: cambios pequeños pueden producir otro árbol;
- sobreajuste si no se controla la complejidad;
- divisiones codiciosas que no garantizan el árbol globalmente óptimo;
- probabilidades por hoja potencialmente poco calibradas;
- sesgos de los datos se trasladan a las reglas y resultados.

Antes de cualquier uso real

Se requieren datos representativos, definición clínica válida del desenlace, tratamiento documentado de faltantes, validación cruzada, evaluación externa, análisis por subgrupos, calibración y revisión ética/profesional. Nada de eso puede inferirse de este caso sintético.

Capítulo 8

Guion breve para exponer

Tiempo	Mensaje sugerido
0–2 min	Un árbol convierte una predicción en una secuencia de preguntas. Presentar raíz, ramas, nodos y hojas.
2–4 min	Distinguir clasificación y regresión. Explicar que nuestro desenlace es binario y simulado.
4–6 min	Mostrar Gini intuitivamente: una hoja pura tiene valor 0; una mezcla 50/50 es más impura.
6–8 min	Explicar separación entrenamiento/prueba y por qué evita evaluar con los mismos casos usados para aprender.
8–11 min	Recorrer una ruta del árbol y mostrar la matriz de confusión. Contrastar exactitud con sensibilidad.
11–13 min	Mostrar sobreajuste, profundidad y poda.
13–15 min	Resumir ventajas, límites y condiciones necesarias para un uso responsable.

8.1. Preguntas frecuentes

¿Gini o entropía? Ambas miden mezcla y suelen dar resultados similares. Conviene comparar dentro de una validación, no escoger por costumbre.

¿Por qué no usar todos los datos para entrenar? Porque necesitamos una estimación del comportamiento ante casos no vistos.

¿La variable más importante es la causa principal? No. La importancia describe el uso predictivo dentro del árbol y los datos observados.

¿Por qué limitar la profundidad? Para controlar sobreajuste, estabilidad y facilidad de interpretación.

¿Un árbol interpretable es automáticamente confiable? No. Las reglas pueden ser transparentes y, al mismo tiempo, estar sesgadas o no generalizar.

Capítulo 9

Reproducir el proyecto

Desde la raíz del repositorio:

```
conda env create -f environment.yml
conda activate arboles-decision
make all
```

El flujo sigue este orden:

1. scripts/generate_data.py crea el CSV de entrada;
2. scripts/run_analysis.py ajusta el árbol y genera métricas, predicciones, reglas, tablas y figuras;
3. Jupyter ejecuta el notebook y guarda las salidas;
4. Tectonic compila esta guía incorporando los resultados generados;
5. las pruebas verifican Gini, la separación y propiedades básicas del modelo.

Una sola fuente de verdad

La lógica del modelo vive en src/arboles_decision/pipeline.py. El notebook, los scripts y el PDF consumen esa misma implementación. Así se reduce el riesgo de presentar números distintos a los que produce el código.

Referencias y lecturas

- Breiman, L., Friedman, J. H., Olshen, R. A., y Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.
- Ma, X. (2018). *Using Classification and Regression Trees: A Practical Primer*. Information Age Publishing. Capítulo 3, pp. 39–47.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Lista de comprobación final

- ☐ Puedo explicar una ruta desde la raíz hasta una hoja.
- ☐ Puedo calcular Gini en un ejemplo pequeño.
- ☐ Distingo entrenamiento, validación y prueba.
- ☐ Interpreto los cuatro cuadrantes de la matriz de confusión.
- ☐ Sé por qué exactitud no siempre es suficiente.
- ☐ Puedo explicar sobreajuste y dos formas de controlarlo.
- ☐ No confundo importancia predictiva con causalidad.
- ☐ Puedo declarar claramente que el caso es sintético y educativo.